SIMATS ENGINEERING

(Saveetha School of Engineering)

CO3 ASSESSMENT TOOL – 2
CODE REVIEW & REPOSITORY EVALUATION

Fraud Detection and Financial Transaction Monitoring System

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Jeyakannan L	**Reg. No.:**	192525376
Date:	17-08-2026	**Duration:**	60 minutes

Assessment Questions Covered

- 1. Problem Overview
- 2. Repository Organization
- 3. Code Quality Review
- 4. Version Control Evaluation
- 5. Code Testing and Validation
- 6. Repository Documentation
- 7. Code Review Findings and Recommendations

Project Context

The project reviewed in this report is a Fraud Detection and Financial Transaction Monitoring System. It is designed as a full-stack application that records financial transactions, monitors suspicious activity, calculates a fraud risk score, generates alerts and provides a dashboard for authorized users.

This report evaluates the repository organization, source-code quality, Git practices, testing evidence and documentation. The recommendations are aligned with the assessment rubric and target the Excellent level.

1. Problem Overview

Problem Statement

Financial institutions process a high volume of transactions, making it difficult to manually identify every suspicious transaction. Fraudulent or unusual transactions can cause financial loss and security risks. The proposed system automates transaction monitoring by analysing transaction information, assigning a risk score and generating alerts for high-risk activity.

Implemented Solution

The application uses a frontend dashboard for user interaction, a backend REST API for application logic, a fraud-detection service for risk analysis and PostgreSQL for persistent storage. Git manages the source-code history, while Docker provides consistent development and deployment environments.

Key Functionalities

- User authentication and authorized access.
- Financial transaction recording and validation.
- Transaction monitoring and filtering.
- Fraud risk scoring.
- Suspicious-transaction alert generation.
- Transaction and alert history.
- Dashboard-based monitoring and reporting.
- REST API communication between application components.

Expected Outcome

The expected outcome is a maintainable and testable application that can process transactions, identify suspicious behaviour and present actionable monitoring information to authorized users.

2. Repository Organization

The repository should follow a modular structure so that the frontend, backend, fraud service, database, tests and documentation are separated. This improves readability, maintainability and collaboration.

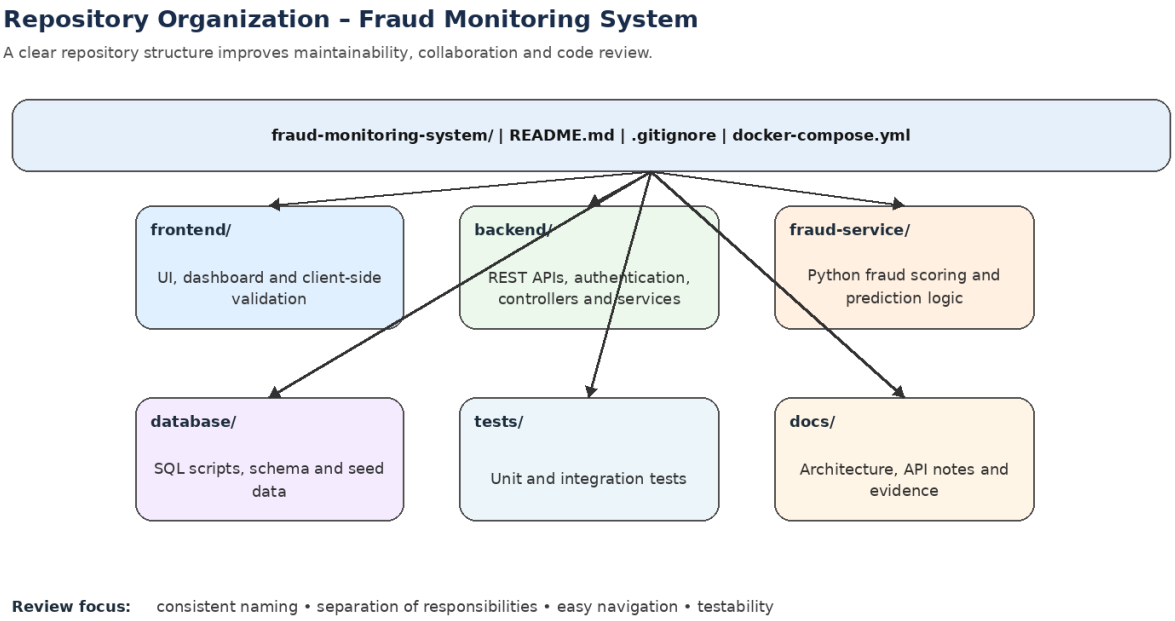

Figure 1. Recommended repository structure for the project.

Evaluation

Folder organization: The major application responsibilities are separated into frontend, backend, fraud-service, database, tests and docs.

Naming conventions: Folder names are descriptive and use consistent lowercase naming.

Separation of concerns: UI, API logic, fraud analysis, persistence and testing are not mixed together.

Collaboration: Developers can work on separate components using feature branches.

Maintainability: A change to one component can normally be made without rewriting unrelated components.

Repository Quality Recommendations

- Keep a clear README.md at the repository root.
- Keep secrets and .env files outside version control.
- Store automated tests close to the component or in a clearly named tests directory.
- Use consistent naming for files, folders, classes, functions and API routes.
- Keep generated files, build output and dependency folders out of Git.

3. Code Quality Review

The code review focuses on readability, modularity, coding standards, documentation and maintainability. The objective is not only to verify whether the application works, but also to determine whether the code can be safely understood, tested and extended.

Readability

- Use descriptive variable and function names such as calculateRiskScore() instead of vague names such as process().
- Keep functions focused on one responsibility.
- Use consistent indentation and formatting.
- Avoid unnecessary nesting and duplicated logic.

Modularity

The backend should separate routes/controllers, services, models and middleware. Fraud-detection logic should remain in the fraud-service instead of being mixed with UI or database code.

Coding Standards

- Use a consistent naming convention throughout the project.
- Validate all API inputs before processing.
- Handle errors using structured responses rather than exposing internal exceptions.
- Keep configuration values in environment variables.
- Use reusable utility functions for repeated operations.

Documentation in Code

Public APIs, non-obvious business rules and important fraud-scoring assumptions should be documented. Comments should explain why a decision is made rather than simply repeating what the code does.

Maintainability Review

The proposed modular architecture supports maintainability because changes can be isolated. For example, changing the fraud-scoring implementation should not require changes to the frontend transaction form.

4. Version Control Evaluation

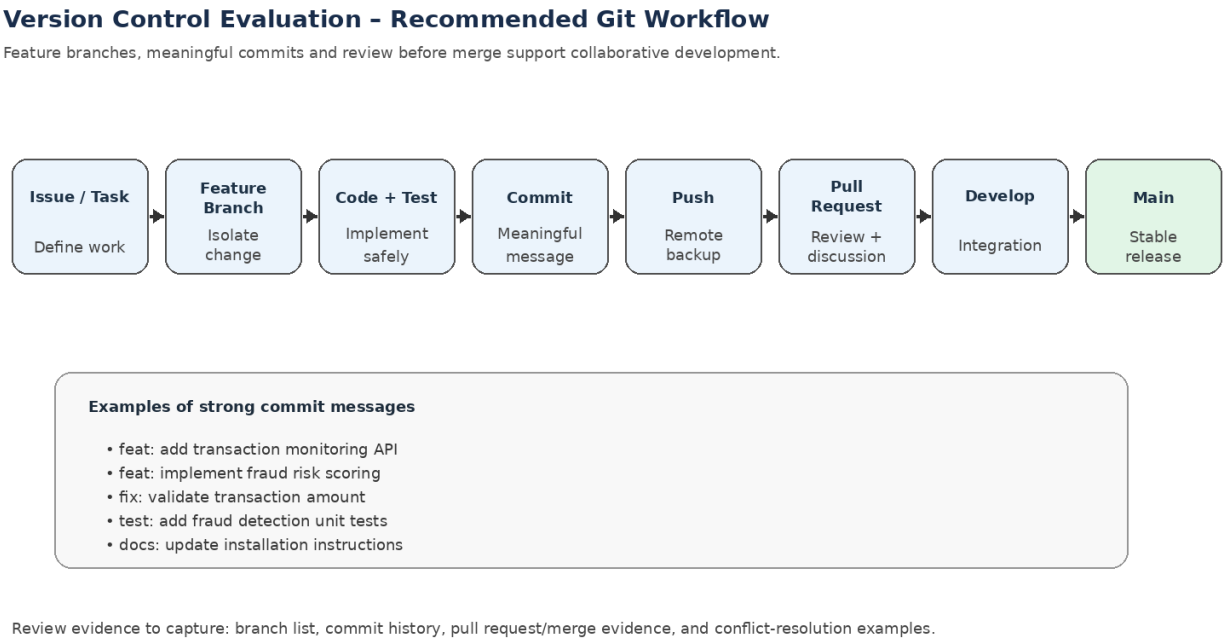

Figure 2. Recommended Git workflow for collaborative development.

Branching Strategy

A main + develop + feature-branch workflow is suitable. main contains stable code, develop contains integrated development work and feature branches isolate individual changes.

```
main
  |
  └── develop
       ├── feature/login
       ├── feature/transaction-monitoring
       ├── feature/fraud-detection
       ├── feature/dashboard
       └── feature/docker
```

Commit Quality

Commit messages should explain the purpose of each change. Examples:

```
feat: add transaction monitoring API
feat: implement fraud risk scoring
fix: validate transaction amount
test: add fraud detection unit tests
docs: update installation instructions
```

Repository Management Evaluation

Commit history: A logical sequence of small, meaningful commits makes project progress traceable.

Branching: Feature branches reduce risk to stable code and support parallel work.

Synchronization: Developers should pull recent changes before starting work and resolve conflicts carefully.

Review: Pull requests should be reviewed before merging into shared branches.

Traceability: Issues, branches, commits and pull requests should reference the same feature or defect where possible.

Example Commit History

```
9f31abc  docs: update deployment documentation
7c21d45  feat: add Docker Compose configuration
5a72c19  feat: implement fraud risk scoring
4b63e10  feat: add transaction monitoring API
3d52a11  feat: create transaction database model
2a41b08  feat: create login functionality
1a30c07  chore: initialize project
```

5. Code Testing and Validation

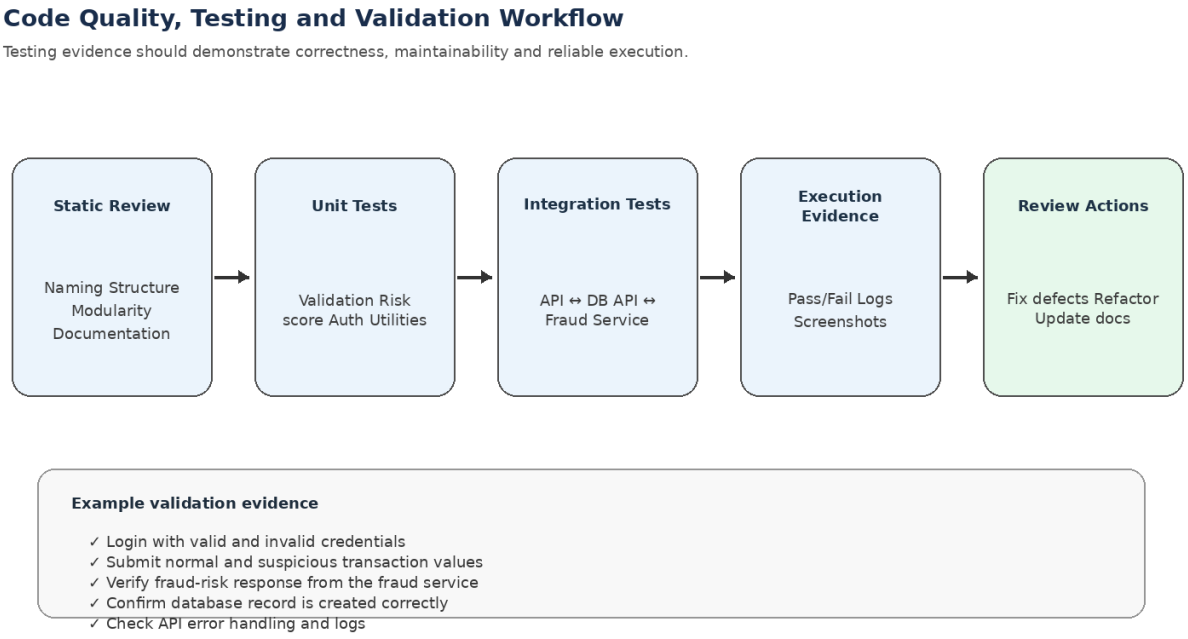

Figure 3. Code-quality and testing validation workflow.

Testing Approach

Testing should cover individual functions, interactions between components and complete user workflows. Execution results and screenshots should be retained as evidence for the assessment.

Suggested Test Cases

Test Case	Input / Action	Expected Result	Evidence
TC01 Login	Valid username/password	User reaches dashboard	Screenshot/log
TC02 Login failure	Invalid credentials	Access denied with safe error	Screenshot
TC03 Transaction validation	Negative/invalid amount	Validation error returned	API result
TC04 Normal transaction	Valid low-risk transaction	Transaction stored successfully	DB/API evidence
TC05 Suspicious transaction	High-risk transaction pattern	High risk score/alert generated	Dashboard screenshot

TC06 Fraud API	Send transaction data	Risk score returned	API response
TC07 Database	Create/read transaction	Correct record retrieved	DB result
TC08 Error handling	Unavailable dependency	Graceful error response	Log screenshot

Testing Evidence

- Unit-test execution output.
- API test results.
- Successful and failed test cases.
- Application screenshots.
- Docker container status and logs.
- Database verification evidence.
- Evidence of corrected defects after review.

Validation Result

A strong submission should demonstrate not only that the application runs, but also that important requirements have been tested with appropriate inputs, expected results and execution evidence.

6. Repository Documentation

README.md Requirements

- Project title and short description.
- Problem statement and objectives.
- Technology stack.
- Repository structure.
- System requirements and prerequisites.
- Installation instructions.
- Environment-variable configuration.
- How to run locally.
- How to run using Docker Compose.
- API endpoints and example requests.
- Testing instructions.
- Known limitations.
- Future improvements.

Example README Structure

```
# Fraud Detection and Financial Transaction Monitoring System

## 1. Project Overview
## 2. Features
## 3. Technology Stack
## 4. Repository Structure
## 5. Installation
## 6. Environment Configuration
## 7. Running the Application
## 8. Docker Deployment
## 9. API Documentation
## 10. Testing
## 11. Screenshots
## 12. Limitations
## 13. Future Improvements
```

Documentation Quality Evaluation

Completeness: README should contain enough information for another student/developer to install and run the project.

Clarity: Instructions should be written as numbered, reproducible steps.

Accuracy: Commands and file paths must match the actual repository.

Maintainability: Documentation should be updated whenever setup, APIs or project structure changes.

Evidence: Architecture diagrams, screenshots and test results should be stored in docs/ or an equivalent location.

7. Code Review Findings and Recommendations

Summary of Review Findings

Area	Finding	Impact	Recommendation
Repository	Clear separation is needed between application components and documentation.	Improves navigation and collaboration.	Maintain frontend/backend/fraud-service/tests/docs structure.
Code Quality	Repeated validation or business logic may become difficult to maintain if duplicated.	Higher defect and maintenance risk.	Move shared logic into reusable services/utilities.
Security	Secrets should not be committed to Git.	Credential exposure risk.	Use .env locally, .gitignore and secure secret management.
Version Control	Large or vague commits make changes difficult to review.	Poor traceability.	Use small, meaningful commits with conventional prefixes.
Testing	Core fraud rules and API interactions need explicit test evidence.	Undetected defects may reach users.	Add unit, integration and negative test cases.
Documentation	README must match actual commands and folder names.	New developers may fail to run the project.	Review README after every major change.
Maintainability	Configuration and dependencies should be isolated from business logic.	Changes become harder to manage.	Use environment configuration and modular services.

Priority Improvements

1. Protect secrets and sensitive configuration.
2. Increase automated test coverage for critical transaction and fraud logic.
3. Maintain a clean and meaningful Git history.
4. Document setup, Docker execution and API usage completely.
5. Apply consistent coding standards and modular design.
6. Add CI checks for tests, formatting and basic code quality.
7. Keep screenshots and execution evidence for the final assessment submission.

Overall Assessment Against Rubric

Rubric Area	Excellent-Level Evidence	Target
Problem Analysis & Organization (15%)	Clear problem, solution, objectives, repository structure and justification.	Excellent

Code Quality Review (25%)	Thorough review of readability, modularity, standards, documentation and maintainability.	Excellent
Version Control Evaluation (20%)	Branching, meaningful commits, commit history and repository-management practices are demonstrated.	Excellent
Testing & Validation (15%)	Appropriate test cases, execution evidence, screenshots and defect validation.	Excellent
Documentation & Repository (15%)	Complete README, setup, usage guide, dependencies and maintainable repository documentation.	Excellent
Review Findings & Presentation (10%)	Specific findings, practical recommendations and professional presentation.	Excellent

Conclusion

The Code Review and Repository Evaluation confirms that the Fraud Detection and Financial Transaction Monitoring System can be maintained effectively when the repository is modular, the code follows consistent standards, Git history is meaningful, testing is supported by evidence and documentation is kept accurate.

The highest-value improvements are automated testing of critical fraud and transaction functions, protection of secrets, disciplined Git workflows, complete README documentation and regular code reviews. These practices directly support the assessment criteria and improve the quality of the final software system.